

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

<https://github.com/seejokim/na-python>

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업 혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
3	선형연립방정식 수치해법	3
4	수치미분 (Numerical Differentiation)	4
5	수치적분 (Numerical Integration)	5
5.1	리만 (Riemann) 적분	5
5.1.1	리만 적분의 정의	5
5.1.2	리만 (Riemann) 적분 가능성	5
5.1.3	리만 합 구현 및 가시화	5
5.2	사다리꼴 적분 (Trapezoidal rule)	6
5.2.1	사다리꼴 적분 (Trapezoidal rule)	6
5.2.2	사다리꼴 적분(Trapezoidal rule) Python 프로그래밍 1	6
5.2.3	사다리꼴 적분(Trapezoidal rule) Python 프로그래밍 2	7
5.3	Richardson 외삽법 (Extrapolation)	7
5.3.1	(5.3.1) Richardson Extrapolation 개요	7
5.3.2	파이썬 코드	9
5.4	Romberg Integration	9
5.4.1	Romberg Integration 원리	9
5.4.2	Romberg Integration Python 프로그래밍	9
5.5	Simpson's 1/3rd Rule	10
5.5.1	Simpson's Rule 개요	10
5.5.2	Simpson's 1/3 Rule 프로그래밍	11
5.6	Newton-Cotes Quadrature	12
5.6.1	설명	12
5.6.2	일반 표현	12
5.6.3	유형별 Newton-Cotes 공식	12
5.7	Gauss Quadrature Rule (가우스 구적법)	13
5.7.1	Gauss Quadrature Rule (가우스 구적법)	13
5.7.2	가우스 구적법의 1중 적분	14
5.7.3	가우스 구적법의 2중 적분	16
5.7.4	가우스 구적법을 활용한 사각형 면적 구하기	17
5.7.5	사각형 면적 구하기 코드	18
5.8	연습문제	18

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

Chapter 3

선형연립방정식 수치해법

Chapter 4

수치미분 (Numerical Differentiation)

Chapter 5

수치적분 (Numerical Integration)

5.1 리만 (Riemann) 적분

5.1.1 리만 적분의 정의

정적분은 다음 극한으로 정의된다.

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i^*) \Delta x \quad (5.1)$$

5.1.2 리만 (Riemann) 적분 가능성

연속함수는 리만 적분 가능하다.

5.1.3 리만 합 구현 및 가시화

아래는 리만 적분을 구현하고 가시화하는 Python 코드이다. 좌측, 우측, 중앙 리만 합을 각각 계산하고 비교하는 방식으로 작성하였다.

1. 구현 방식

- (a) 함수 $f(x) = x^2$ 를 정의하고, 구간 $[0, 2]$ 에서 적분을 계산.
- (b) 리만 합의 세 가지 방법(좌측, 우측, 중앙)을 각각 구현.

2. 결과

- (a) 좌측 리만 합과 우측 리만 합은 구간의 시작과 끝에 따라 오차가 존재.
- (b) 중앙 리만 합은 오차가 줄어들며 더 정확한 결과 제공.

3. 가시화

- (a) 리만 합 계산 과정에서 사용된 직사각형이 시각적으로 표현됨.
- (b) 좌측, 중앙, 우측 리만 합에 따라 직사각형 높이가 달라짐을 확인 가능.

4. 활용

- (a) 리만 적분은 수치적분의 기초로, 사다리꼴 법칙이나 심프슨 법칙과 같은 고급 적분법의 기초가 된다.

- (b) 머신러닝에서는 데이터 분포를 근사하거나 손실 함수의 값을 계산할 때 유용하게 사용된다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_01_03_riemann_sum_visualization.py

```
import numpy as np

def riemann_sum(f, a, b, n):
    x = np.linspace(a, b, n+1)
    dx = (b - a)/n
    return sum(f(x[i])*dx for i in range(n))
```

5.2 사다리꼴 적분 (Trapezoidal rule)

5.2.1 사다리꼴 적분 (Trapezoidal rule)

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right] \quad (5.2)$$

5.2.2 사다리꼴 적분(Trapezoidal rule) Python 프로그래밍 1

1. 이론값과 사다리꼴 공식을 통한 값 비교

이론적으로 풀이한 값

$$E_t = -\frac{1}{12} f''(\xi)(b-a)^3 \quad (5.2.2-1)$$

$$\int_0^{10} (x+3) dx = \left[\frac{1}{2}x^2 + 3x \right]_0^{10} = 80$$

100번 나눈 사다리꼴 공식을 통한 풀이

$$I = \frac{(10-0)}{2 \cdot 100} \left\{ f(x_0) + 2 \sum_{i=1}^{99} f(x_i) + f(x_{100}) \right\}$$

파이썬을 사용하여 위 식을 계산한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_02_02_trapezoidal_rule_basic.py

```
def trapezoidal(f, a, b, n):
    h = (b - a)/n
    s = 0.5*(f(a)+f(b))
    for i in range(1,n):
        s += f(a+i*h)
    return h*s
```

5.2.3 사다리꼴 적분(Trapezoidal rule) Python 프로그래밍 2

1. 함수를 정의.

```
def y( x ):
    return (1 / (1 + x * x))
```

2. 사다리꼴 함수를 정의. (구간1, 구간2, 나누는 횟수)

```
def trapezoidal (a, b, n):
    h = (b - a) / n
    s = y(a) + y(b)
    i = 1
    while i < n:
        s += 2 * y(a + i * h)
        i += 1
    return (h / 2) * s
```

3. 구간과 나누는 횟수를 입력한 뒤 출력.

```
x0 = 0
xn = 1
n = 6
print ("Value of integral is ",
        "%0.4f"%trapezoidal(x0, xn, n))
```

4. OUTPUT

```
Value of integral is  0.7842
```

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_02_03_trapezoidal_rule_loop.py

NumPy 벡터화 버전

```
def trapezoidal_np(f, a, b, n):
    x = np.linspace(a,b,n+1)
    y = f(x)
    h = (b-a)/n
    return h*(0.5*y[0]+sum(y[1:-1])+0.5*y[-1])
```

5.3 Richardson 외삽법 (Extrapolation)

5.3.1 (5.3.1) Richardson Extrapolation 개요

Richardson 외삽법은 서로 다른 간격 h (또는 분할수 N)에서 계산한 근사값을 결합하여 더 높은 차수의 정확도를 얻는 방법이다. 수치적분, 수치미분에서 널리 사용된다.

1. 오차의 형태

사다리꼴 적분의 오차는 다음과 같이 표현된다.

$$E_N = \frac{C}{N^2}$$

즉, 오차는 $O(N^{-2})$ 로 감소한다.

적분의 정확한 값 $I = \int_a^b f(x) dx$ 라 하면,

$$I = I_N + \frac{C}{N^2}$$

$$I = I_{2N} + \frac{C}{(2N)^2}$$

2. 두 근사값의 차이

위 두 식의 차를 취하면,

$$I_{2N} - I_N = \frac{C}{N^2} - \frac{C}{(2N)^2} = \frac{3C}{4N^2}$$

따라서,

$$\frac{C}{N^2} = \frac{4}{3}(I_{2N} - I_N)$$

3. Richardson 외삽 공식

이를 이용하면 오차를 제거한 더 정확한 값은

$$I \approx \frac{4I_{2N} - I_N}{3}$$

즉,

$$I_{\text{extrapolated}} = \frac{4I_{2N} - I_N}{3}$$

이 식은 $O(N^{-4})$ 정확도를 갖는다.

4. 핵심 정리

- 오차가 $O(N^{-2})$ 일 때 Richardson 적용 가능
- 서로 다른 격자 간격 결과를 결합하여 오차 제거
- 정확도 차수 상승: $O(N^{-2}) \rightarrow O(N^{-4})$
- Romberg 적분의 기본 원리

$$I \approx \frac{4I(h/2) - I(h)}{3} \tag{5.3}$$

5.3.2 파이썬 코드

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_03_02_richardson_extrapolation.py

```
def richardson_trap(f,a,b,n):
    I1 = trapezoidal(f,a,b,n)
    I2 = trapezoidal(f,a,b,2*n)
    return (4*I2 - I1)/3
```

5.4 Romberg Integration

5.4.1 Romberg Integration 원리

Romberg 방법은 Richardson 외삽을 반복 적용한 것이다.

Romberg 적분은 사다리꼴 공식과 Richardson 외삽법을 결합하여 정확도를 체계적으로 향상시키는 방법이다.

1. 사다리꼴 공식

구간 $[a, b]$ 를 N 등분하면,

$$I_N = \int_a^b f(x) dx \approx \frac{\Delta x}{2} \left[f(a) + 2 \sum_{k=1}^{N-1} f(a + k\Delta x) + f(b) \right]$$

여기서 $\Delta x = \frac{b-a}{N}$ 이다.

2. 분할을 두 배로 증가시키면

$N \rightarrow 2N$ 이면,

$$\Delta x \rightarrow \frac{\Delta x}{2}$$

이때 적분은

$$I_{2N} = \frac{\Delta x}{4} \left[f(a) + 2 \sum_{k=1}^{2N-1} f\left(a + k\frac{\Delta x}{2}\right) + f(b) \right]$$

이를 정리하면,

$$I_{2N} = \frac{1}{2} I_N$$

5.4.2 Romberg Integration Python 프로그래밍

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_04_02_romberg_integration.py

```

def romberg(f,a,b,n):
    R = np.zeros((n,n))
    for i in range(n):
        R[i,0] = trapezoidal(f,a,b,2**i)
        for k in range(1,i+1):
            R[i,k] = (4**k*R[i,k-1] - R[i-1,k-1]) / (4**k - 1)
    return R[n-1,n-1]

```

5.5 Simpson's 1/3rd Rule

5.5.1 Simpson's Rule 개요

Simpson의 법칙은 구간 $[a, b]$ 에서 함수 $f(x)$ 를 2차 다항식으로 근사하여 적분값을 구하는 방법이다.

1. 2차 보간 다항식 근사

구간 $[a, b]$ 에서

$$x_0 = a, \quad x_1 = \frac{a+b}{2}, \quad x_2 = b$$

2차 다항식

$$p_2(x) = a_0 + a_1x + a_2x^2$$

로 근사하면

$$\begin{aligned}
 A &= \int_a^b p_2(x) dx \\
 &= \left[a_0x + \frac{a_1}{2}x^2 + \frac{a_2}{3}x^3 \right]_a^b
 \end{aligned}$$

2. Simpson의 1/3 공식

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

또는

$$I = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)]$$

where

$$h = \frac{b-a}{2}$$

3. 오차항

$$E = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

Simpson 공식은 $O(h^4)$ 정확도를 가진다.

4. 합성 Simpson 공식 (Composite Simpson's Rule)

구간을 n 개의 짝수 분할 (n even)로 나누면,

$$h = \frac{b-a}{n}$$

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i=1,3,\dots}^{n-1} f(x_i) + 2 \sum_{i=2,4,\dots}^{n-2} f(x_i) + f(x_n) \right]$$

특징

- 2차 다항식 보간 기반
- 사다리꼴 공식보다 높은 정확도
- n 은 반드시 짝수
- 정확도: $O(h^4)$

5.5.2 Simpson's 1/3 Rule 프로그래밍

다음 적분을 계산한다.

$$\int_0^{10} (x+3) dx$$

1. 이론적 해

$$\int_0^{10} (x+3) dx = \left[\frac{1}{2}x^2 + 3x \right]_0^{10} = 80$$

2. 사다리꼴 공식 (n=100)

$$I_T = \frac{10-0}{2 \cdot 100} \left[f(x_0) + 2 \sum_{i=1}^{99} f(x_i) + f(x_{100}) \right]$$

3. Simpson's 1/3 공식 (n=100, even)

$$I_S = \frac{10-0}{3 \cdot 100} \left[f(x_0) + 4 \sum_{i=1}^{50} f(x_{2i-1}) + 2 \sum_{i=1}^{49} f(x_{2i}) + f(x_{100}) \right]$$

비교 목적

- 이론값: 80
- 사다리꼴 공식 결과
- Simpson 공식 결과

Simpson 공식이 사다리꼴 공식보다 더 높은 정확도를 보인다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_05_02_simpson_one_third_rule.py

```
def simpson(f,a,b,n):
    if n%2==1:
        raise ValueError("n must be even")
    h=(b-a)/n
    s=f(a)+f(b)
    for i in range(1,n):
        coeff=4 if i%2==1 else 2
        s+=coeff*f(a+i*h)
    return h*s/3
```

5.6 Newton-Cotes Quadrature

5.6.1 설명

Newton-Cotes 공식은 등간격 점에서 보간다항식을 이용하여 적분을 근사한다.

5.6.2 일반 표현

$$\int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i) \quad (5.4)$$

5.6.3 유형별 Newton-Cotes 공식

Newton-Cotes 공식은 구간 $[a, b]$ 에서 함수를 다항식으로 근사하여 적분을 계산하는 방법이다.

1. 사다리꼴 법칙 (Trapezoidal Rule, $n = 1$)

1차 다항식 근사:

$$\int_a^b f(x) dx = \frac{b-a}{2} [f(a) + f(b)] \quad (5.6.3-1)$$

2. Simpson 법칙 (Simpson's Rule, $n = 2$)

2차 다항식 근사:

$$\int_a^b f(x) dx = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \quad (5.6.3-2)$$

3. Simpson 3/8 법칙 ($n = 3$)

3차 다항식 근사:

$$\int_a^b f(x) dx = \frac{3(b-a)}{8} [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)] \quad (5.6.3-3)$$

4. Boole 법칙 (4차 Newton–Cotes, $n = 4$)

4차 다항식 근사:

$$\int_a^b f(x) dx = \frac{2(b-a)}{45} [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)] \quad (5.6.3-4)$$

정리

- n 이 증가할수록 근사 다항식 차수가 증가
- 일반적으로 정확도 향상
- 그러나 고차 다항식은 진동(Runge 현상) 가능성 존재

5.7 Gauss Quadrature Rule (가우스 구적법)**5.7.1 Gauss Quadrature Rule (가우스 구적법)**

가우스 구적법(Gaussian Quadrature)은 정적분을 고정된 가중치와 절점(node)을 이용하여 높은 정확도로 근사하는 방법이다. 함수를 단순 다항식으로 근사하지 않고, 최적의 절점과 가중치를 선택한다.

1. 구간 변환

적분 구간 $[a, b]$ 를 표준 구간 $[-1, 1]$ 로 변환한다.

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2}\xi + \frac{a+b}{2}\right) d\xi \quad (5.7.1-1)$$

2. 기본 가우스 근사식

$$\int_{-1}^1 f(\xi) d\xi \approx \sum_{i=1}^n w_i f(\xi_i) \quad (5.7.1-2)$$

- ξ_i : 절점 (nodes) - w_i : 가중치 (weights) - n : 절점 개수

3. 2점 Gauss–Legendre 예

$$\int_{-1}^1 f(\xi) d\xi \approx w_1 f(\xi_1) + w_2 f(\xi_2) \quad (5.7.1-3)$$

2점 가우스 구적법은 **최대 3차 다항식까지 정확히 적분**한다.
조건식 예:

$$\int_{-1}^1 1 d\xi = 2 = w_1 + w_2$$

$$\int_{-1}^1 \xi d\xi = 0 = w_1 \xi_1 + w_2 \xi_2$$

$$\int_{-1}^1 \xi^2 d\xi = \frac{2}{3} = w_1 \xi_1^2 + w_2 \xi_2^2$$

4. 대표적인 Gauss-Legendre 값

- 1점:

$$\xi_1 = 0, \quad w_1 = 2$$

- 2점:

$$\xi_{1,2} = \pm \frac{1}{\sqrt{3}}, \quad w_{1,2} = 1$$

- 3점:

$$\xi = 0, \pm 0.7745966692$$

$$w = 0.8888888889, 0.5555555556$$

핵심 특징

- 동일한 절점 수 대비 Newton-Cotes 방법보다 높은 정확도
- 다항식 차수 $2n - 1$ 까지 정확
- 고차 다항식 또는 매끄러운 함수 적분에 매우 효과적
- 수치해석 및 유한요소법(FEM)에서 널리 사용

5.7.2 가우스 구적법의 1중 적분

가우스 구적법(Gauss Quadrature Rule)은 일반 구간 $[x_0, x_1]$ 의 적분을 표준 구간 $[-1, 1]$ 으로 변환하여 계산한다.

1. 구간 변환

구간 변환:

$$x = \frac{x_1 - x_0}{2} \xi + \frac{x_1 + x_0}{2}$$

Jacobian:

$$dx = \frac{x_1 - x_0}{2} d\xi$$

따라서 적분식은

$$\int_{x_0}^{x_1} \phi(x) dx = \frac{x_1 - x_0}{2} \int_{-1}^1 \phi\left(\frac{x_1 - x_0}{2} \xi + \frac{x_1 + x_0}{2}\right) d\xi \quad (5.7.2-1)$$

2. Gauss-Legendre 공식

$$\int_{-1}^1 f(\xi) d\xi \approx \sum_{i=1}^n w_i f(\xi_i)$$

따라서 일반 구간에서

$$\int_{x_0}^{x_1} \phi(x) dx \approx \frac{x_1 - x_0}{2} \sum_{i=1}^n w_i \phi\left(\frac{x_1 - x_0}{2} \xi_i + \frac{x_1 + x_0}{2}\right) \quad (5.7.2-2)$$

3. 예제 1

$$\int_1^2 (3x + 2) dx$$

구간 변환:

$$x = \frac{2-1}{2}\xi + \frac{2+1}{2} = \frac{1}{2}\xi + \frac{3}{2}$$

직접 계산 결과:

$$\int_1^2 (3x + 2) dx = \frac{13}{2} \quad (5.7.2-3)$$

4. 예제 2 (n=2 Gauss)

$n = 2$ 일 때:

$$\xi_1 = -\frac{1}{\sqrt{3}}, \quad \xi_2 = \frac{1}{\sqrt{3}}$$

$$w_1 = w_2 = 1$$

따라서

$$\int_{x_0}^{x_1} f(x) dx \approx \frac{x_1 - x_0}{2} [f(x(\xi_1)) + f(x(\xi_2))] \quad (5.7.2-4)$$

핵심 정리

- 적분 구간을 $[-1, 1]$ 로 변환
- 가중치 w_i 와 적분점 ξ_i 사용
- 다항식 차수가 낮을수록 정확도 높음
- n 증가 $\rightarrow$ 고차 다항식까지 정확

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_07_02_gauss_quadrature_1d.py

5.7.3 가우스 구적법의 2중 적분

1. 2D 가우스 구적 기본식

2차원 적분을 표준 구간 $[-1, 1] \times [-1, 1]$ 에서 다음과 같이 근사한다.

$$\iint \phi(x, y) dx dy = \int_{-1}^1 \int_{-1}^1 \phi(\xi, \eta) |J| d\xi d\eta$$

이를 n 점 가우스 구적법으로 확장하면,

$$\iint \phi(x, y) dx dy \approx \sum_{i=1}^n \sum_{j=1}^n w_i w_j \phi(\xi_i, \eta_j) |J(\xi_i, \eta_j)|$$

여기서

- ξ_i, η_j : 가우스 적분점
- w_i, w_j : 가중치
- $|J|$: Jacobian 행렬식

—

2. 형상함수 (Shape Function)

사각형 요소에서 자연좌표 (s, t) 를 사용한다.

$$N_1 = \frac{(1-s)(1-t)}{4}, \quad N_2 = \frac{(1+s)(1-t)}{4}$$

$$N_3 = \frac{(1+s)(1+t)}{4}, \quad N_4 = \frac{(1-s)(1+t)}{4}$$

물리 좌표는 다음과 같이 표현된다.

$$x = \sum_{i=1}^4 N_i x_i, \quad y = \sum_{i=1}^4 N_i y_i$$

—

3. Jacobian 행렬

좌표 변환 행렬:

$$J = \begin{bmatrix} \frac{\partial x}{\partial s} & \frac{\partial x}{\partial t} \\ \frac{\partial y}{\partial s} & \frac{\partial y}{\partial t} \end{bmatrix}$$

행렬식:

$$|J| = \frac{\partial x}{\partial s} \frac{\partial y}{\partial t} - \frac{\partial x}{\partial t} \frac{\partial y}{\partial s}$$

형상함수를 이용하면,

$$\frac{\partial x}{\partial s} = \sum_{i=1}^4 \frac{\partial N_i}{\partial s} x_i, \quad \frac{\partial y}{\partial t} = \sum_{i=1}^4 \frac{\partial N_i}{\partial t} y_i$$

4. 핵심 정리

- 2D 적분은 1D 가우스 구적의 곱 형태로 확장된다.
- 형상함수를 통해 물리 좌표 $\leftrightarrow$ 자연좌표 변환을 수행한다.
- Jacobian 행렬식은 면적 변환 계수 역할을 한다.
- 최종 적분식은

$$\sum_{i=1}^n \sum_{j=1}^n w_i w_j \phi(\xi_i, \eta_j) |J|$$

형태로 계산된다.

5. 최종 Jacobian 행렬식 정리

Jacobian 행렬식은 다음과 같이 정리된다.

$$\det J = |J| = \frac{\partial x}{\partial s} \frac{\partial y}{\partial t} - \frac{\partial x}{\partial t} \frac{\partial y}{\partial s}$$

형상함수를 대입하여 정리하면,

$$\det J = \frac{1}{8} [(x_1 - x_3)(y_2 - y_4) - (x_2 - x_4)(y_1 - y_3)] \quad (5.7.3-10)$$

5.7.4 가우스 구적법을 활용한 사각형 면적 구하기

1. 사각형 면적의 기본식

사각형 요소의 면적은 표준 영역 $[-1, 1] \times [-1, 1]$ 로 변환하여 계산한다.

$$A = \iint_{\Omega} dx dy = \int_{-1}^1 \int_{-1}^1 |J| ds dt \approx \sum_{i=1}^n \sum_{j=1}^n w_i w_j |J| \quad (5.7.4-1)$$

여기서 J 는 Jacobian 행렬이다.

2. Jacobian 행렬식

사각형 4개 절점 (x_i, y_i) 에 대해

$$\det J = |J| = \frac{1}{8} [(x_1 - x_3)(y_2 - y_4) - (x_2 - x_4)(y_1 - y_3)] \quad (5.7.4-2)$$

이는 실제 좌표와 표준 좌표 간의 면적 스케일링 계수이다.

3. 가우스 구적 적용

(1) 1점 가우스 적분

$$\int_{-1}^1 \int_{-1}^1 |J| ds dt \approx w_1 w_1 |J|$$

$$w_1 = 2, \quad s_1 = 0, \quad t_1 = 0$$

(2) 2점 가우스 적분

$$\int_{-1}^1 \int_{-1}^1 |J| ds dt \approx \sum_{i=1}^2 \sum_{j=1}^2 w_i w_j |J|$$

$$w_1 = w_2 = 1$$

$$s_1 = -0.5774, \quad s_2 = +0.5774$$

$$t_1 = -0.5774, \quad t_2 = +0.5774$$

핵심 요약

- 사각형 면적은 $\det J$ 를 이용하여 표준 영역에서 계산
- 1점 구적은 계산이 단순하지만 정확도 낮음
- 2점 구적은 일반 사각형 요소 면적 계산에 충분히 정확
- FEM에서 요소 면적 및 적분 계산의 기본 구조

5.7.5 사각형 면적 구하기 코드

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter05/section_05_07_04_gauss_quadrature_rectangle_area.py

```
import numpy as np
from numpy.polynomial.legendre import leggauss

def gauss_quadrature(f,a,b,n):
    x,w=leggauss(n)
    t=0.5*(x+1)*(b-a)+a
    return 0.5*(b-a)*sum(w*f(t))
```

5.8 연습문제